PDF RAG CHATBOT TEST DOCUMENT

Introduction to Artificial Intelligence and Machine Learning

A practical, text-based guide for testing document retrieval, embeddings, FAISS search, and grounded answers.

Document type: Educational reference guide
Intended use: Demonstration and testing of a Retrieval-Augmented Generation chatbot
Topics: AI, machine learning, evaluation, embeddings, RAG, privacy, deployment

This document is original sample content prepared for RAG testing. It is not a scientific paper and does not claim to provide exhaustive coverage of the field.

Common Types of Machine Learning

Supervised learning

Supervised learning uses examples that include both input features and target labels. A model learns to predict the target from the features. Classification predicts a category, such as whether an email is spam. Regression predicts a numeric value, such as the expected price of a house.

Unsupervised learning

Unsupervised learning works with data that has no provided target labels. Clustering groups similar observations, while dimensionality-reduction methods represent data with fewer variables. These methods can help explore structure, but the resulting groups still require interpretation.

Reinforcement learning

Reinforcement learning studies how an agent can choose actions in an environment to maximize a cumulative reward. The agent observes a state, takes an action, receives feedback, and updates its strategy. It is used in areas such as simulated control and sequential decision-making.

Choosing an approach

The task and available data guide the choice. If labelled examples are available and the goal is prediction, supervised learning may fit. If the goal is to discover structure, unsupervised methods may help. If decisions unfold over time and feedback depends on actions, reinforcement learning may be relevant.

Practical caution

A learning method does not guarantee a useful result. Establish a baseline, define success before experimentation, and check whether the data represents the situations in which the model will be used.

The Machine Learning Workflow

1. Define the problem

Describe the decision or outcome the system should support. Specify the prediction target, intended users, acceptable error, and constraints. A vague goal such as “use AI to improve business” is difficult to evaluate; a concrete goal such as “flag potentially fraudulent transactions for human review” is testable.

2. Collect and understand data

Identify the data source, permissions, collection process, and limitations. Inspect row counts, feature types, missing values, duplicates, class balance, and time coverage. Data collection should respect privacy, security, consent, and applicable law.

3. Prepare the data

Common steps include correcting data types, handling missing values, encoding categories, scaling selected numeric features, and creating meaningful features. Fit transformations using the training set only, then apply the fitted transformations to validation and test data to avoid leakage.

4. Train and compare

Start with a simple baseline before testing more complex models. Keep experiments reproducible by recording the data version, features, model settings, random seed when relevant, and evaluation results.

5. Evaluate and deploy

Use a held-out test set for a final estimate of generalization. Before deployment, review reliability, latency, cost, security, and user impact. Monitor performance after release because real-world data can change.

Data Quality, Cleaning, and Leakage

Missing and inconsistent values

Missing values may indicate an ordinary data-entry gap or a meaningful event. Investigate why values are missing before choosing imputation or exclusion. Standardize units, dates, category spelling, and identifiers so that equivalent values are treated consistently.

Duplicates and outliers

Duplicates can overweight repeated observations, while outliers may represent errors or legitimate rare events. Do not automatically remove every extreme value. Inspect context and document the rule used to handle it.

Data leakage

Leakage occurs when training uses information that would not legitimately be available at prediction time. For example, a model predicting whether a customer will cancel should not use a field created after cancellation. Leakage can make offline results look excellent while real-world performance is poor.

Train, validation, and test sets

The training set is used to fit model parameters. The validation set supports model selection and tuning. The test set is reserved for final evaluation. For time-dependent problems, split by time; for related records, keep related entities in the same split where appropriate.

Checklist

Ask: Was each feature available at prediction time? Were preprocessing steps fitted only on training data? Does the test set reflect the intended deployment population? Are duplicate people, devices, or events crossing split boundaries?

Algorithms: Linear Models and Trees

Linear regression

Linear regression predicts a numeric target using a weighted combination of features. It is often useful as an interpretable baseline. Its assumptions and residual patterns should be checked, and relationships that are strongly nonlinear may require feature transformations or another model.

Logistic regression

Despite its name, logistic regression is commonly used for classification. It estimates class probabilities through a logistic function and can provide a relatively interpretable baseline. Probability calibration and the chosen decision threshold matter when false positives and false negatives have different costs.

Decision trees

A decision tree repeatedly splits data using feature-based rules. Trees can represent nonlinear relationships and are easy to explain when small, but a deep tree may overfit. Pruning or limiting tree depth can improve generalization.

Random forests

A random forest combines predictions from many decision trees trained with randomness in samples and feature selection. This often improves stability compared with a single tree, though the ensemble may be less straightforward to explain.

Model choice

No algorithm is universally best. Compare models using the same split and metrics, consider operational constraints, and prefer the simplest model that meets the requirements.

Neural Networks and Deep Learning

Basic structure

A neural network contains layers of units connected by learned weights. Each unit combines inputs, applies an activation function, and passes a result forward. During training, an optimization algorithm adjusts weights to reduce a loss function.

Training loop

A typical training loop performs a forward pass, computes loss, calculates gradients through backpropagation, and updates parameters. Learning rate, batch size, architecture, data quality, and regularization all affect training behaviour.

Overfitting

Overfitting occurs when a model learns details of training examples that do not generalize. Warning signs include strong training performance paired with substantially weaker validation performance. More representative data, regularization, early stopping, or a simpler model may help.

Deep learning applications

Deep neural networks are used in image recognition, speech processing, language modelling, and other high-dimensional tasks. They may require substantial compute and data, and their outputs should be evaluated for robustness and failure modes.

Generative models

Generative models learn patterns in data and produce new outputs. A language model generates text token by token based on context. Fluent output is not proof of factual correctness; generated claims should be grounded and verified in applications where accuracy matters.

Evaluating Model Performance

Classification metrics

Accuracy is the fraction of predictions that are correct. Precision measures the fraction of predicted positives that are truly positive. Recall measures the fraction of actual positives detected. The F1 score is the harmonic mean of precision and recall.

Confusion matrix

A binary confusion matrix counts true positives, true negatives, false positives, and false negatives. It makes error types visible. In a medical screening scenario, for example, missing a true case may have a different cost from sending a healthy person for additional review.

Regression metrics

Mean absolute error (MAE) averages the absolute differences between predictions and observed values. Mean squared error (MSE) squares errors before averaging, giving large errors more influence. Root mean squared error (RMSE) is the square root of MSE and uses the target's units.

Thresholds and imbalance

Accuracy can be misleading when one class is rare. Precision-recall analysis, class-specific metrics, and threshold selection may provide a better view. Choose thresholds based on the real cost of different mistakes, not just a default value.

Reliable comparisons

Evaluate models on the same data and report the metric, split strategy, and relevant uncertainty. A single score without context is rarely enough to judge whether a model is useful.

Embeddings, Vector Search, and RAG

What is an embedding?

An embedding is a numeric representation of content. Embedding models map text into vectors so that semantically related passages may be close in vector space. The quality of retrieval depends on the embedding model, text preparation, chunking, and the content itself.

Chunking

Long documents are divided into smaller text chunks because retrieval systems typically search over manageable passages. Chunk size controls how much context each passage contains; overlap helps preserve information that crosses chunk boundaries. Very small chunks may lose context, while very large chunks may retrieve irrelevant material.

FAISS

FAISS is a library for efficient similarity search over dense vectors. A RAG application can store document-chunk embeddings in a FAISS index and retrieve nearby vectors for a user query. The index is a retrieval structure, not a language model and not a fine-tuned model.

Retrieval-augmented generation

RAG typically embeds or otherwise represents a question, retrieves relevant passages, and supplies those passages to a language model as context. The model then generates an answer grounded in the retrieved material. Citations can help users inspect the source passages.

Common failure modes

A RAG system can fail if the PDF text extraction is poor, relevant content is not retrieved, chunks omit necessary context, or the generator ignores the evidence. Evaluate retrieval quality separately from answer quality, and test questions with known answers.

Responsible AI, Privacy, and Security

Privacy and access

Only use data that you are permitted to process. Avoid sending confidential documents to external APIs without authorization. Restrict access to stored files and indexes, protect credentials, and remove sensitive information when it is not needed.

Bias and representativeness

Historical data can reflect unequal access, measurement errors, or biased decisions. Evaluate performance across relevant groups when appropriate and lawful. A single overall metric can hide important differences between populations.

Human oversight

High-impact decisions should have appropriate human review, escalation paths, and a way to correct errors. Users should understand the system's role and limitations, especially when outputs may affect health, finances, education, employment, or legal rights.

Security basics

Keep API keys in environment variables or a secrets manager rather than source code. Validate uploads, limit file size, handle parsing errors, and avoid executing instructions found inside retrieved documents. Treat document text as data, not as trusted system instructions.

Documentation

Record intended use, excluded uses, data sources, evaluation results, known limitations, and monitoring plans. Clear documentation helps maintainers understand when a model should be trusted and when additional checks are needed.

Deployment, Monitoring, and RAG Test Questions

Deployment checklist

Before release, verify dependency versions, environment variables, file paths, permissions, and startup behaviour. Test the app from a clean environment. For hosted deployments, configure secrets in the hosting platform and confirm that required PDFs and indexes are available at runtime.

Cost and performance

Embedding a document usually creates an initial API cost. Rebuilding the index repeatedly can repeat that cost, so persist or cache indexes where appropriate and invalidate them when the source document or embedding model changes. Each question may incur retrieval and language-model costs based on the prompt and generated answer.

Monitoring

Track latency, errors, token usage, retrieval results, and user feedback while respecting privacy. Test the system after changing the PDF, chunk size, embedding model, prompt, or language model. A change that improves one question may harm another.

Sample questions with grounded answers

Q: What is data leakage? A: It is the use of information during training that would not legitimately be available at prediction time. Q: What does FAISS do in a RAG pipeline? A: It supports efficient similarity search over vectors. Q: Does RAG fine-tune a language model? A: Not by itself; it retrieves external context for generation.

Further testing prompts

Ask the chatbot to compare supervised and unsupervised learning, define precision and recall, explain why a large PDF can increase indexing cost, identify two causes of RAG failure, and list three security practices for API keys and uploaded documents.

Conclusion

A dependable AI application requires more than selecting a model. Clear objectives, high-quality data, sound evaluation, relevant retrieval, privacy safeguards, and ongoing monitoring all contribute to useful outcomes. This sample guide provides a compact document for exercising a PDF-based RAG workflow.